



CDMOROZOV

/ meeting-rooms

Конкурс UA-SKILLS · змагання для junior-розробників · проект «Бронювання переговорних»

ТИП: НА МЕЖІ ПРИЗІВ

PUBLIC-БАЛ: 100 / 100

ФУНКЦІОНАЛ 38,67 / 40

КОД 23 / 25

UI/UX 14,67 / 20

README 12 / 15

01 ПІДСУМОК

Підсумковий публічний бал: **100 зі 100** (стеля шкали). Це сильний результат: робота тримається у верхній частині загального рейтингу учасників, зовсім трохи не доходячи до самої верхньої групи. В основі результату - жива перевірка, де майже всі обов'язкові й бонусні сценарії з ТЗ пройшли без жодного збою: чесний захист від гонки при одночасному бронюванні на рівні бази даних, акуратна робота з часовими поясами і робочими годинами офісу, повторювані серії бронювань, фільтр за місткістю і мобільна розкладка - усе підтверджено вживу, а не тільки заявлено в README.

02 БАЛИ ЗА КАТЕГОРІЯМИ ТЗ

КАТЕГОРІЯ	ВАГА	БАЛ
Функціональність	40	38.67
Код і архітектура	25	23
UI/UX	20	14.67
README та історія розробки	15	12

Сума за чек-листом специфікації: 88.34 зі 100. Фінальний публічний бал (100) включає додаткові бали за реалізований понад обов'язковий мінімум функціонал - подробиці в розділі "Що працює".

03 ЩО ПРАЦЮЄ

Вживу підтверджено такі сценарії:

- Вхід за email і паролем, сесія переживає повне перезавантаження сторінки і переходить між розділами; вихід повертає на екран входу. Невірний пароль дає зрозуміле повідомлення про помилку. Email при вході і реєстрації нормалізується (обрізання пробілів, нижній регістр), повторна реєстрація тієї самої адреси віддає 409.
- Тижнева сітка розкладу: півгодинні ряди, робоче вікно 09:00-19:00, зайняті слоти показують назву і автора, своє бронювання і чуже візуально відрізняються кольором, є лінія поточного часу. Перемикання тижнів коректно проходить межі місяця.

- Створення бронювання через модальне вікно - нове бронювання одразу з'являється в сітці без перезавантаження сторінки. Сусідні за часом бронювання (кінець одного збігається з початком іншого) створюються без хибного конфлікту, а перетин часу блокується і на клієнті, і на сервері зі зрозумілим повідомленням українською.
- Робочі години перевіряються з точністю до хвилини в обидва боки межі: слот, що починається рівно о 09:00 за Києвом, приймається, той, що починається на півгодини раніше, - відхиляється; те саме на закритті о 19:00.
- Бронювання в минулому відхиляється окремим зрозумілим повідомленням, кратність півгодині і межі тривалості (30 хвилин - 4 години) перевіряються з обох боків.
- Скасування бронювання: діалог підтвердження називає конкретне бронювання і попереджає про незворотність дії, слот у сітці звільняється одразу ж; те саме підтвердження працює з розділу "Мої бронювання".
- Розділ "Мої бронювання": вкладки майбутніх і минулих бронювань з правильним сортуванням, клік по рядку переносить у сітку на потрібний тиждень з обраним днем. Курсорна пагінація минулих бронювань перевірена напругу і дійсно працює - у поточному демо-середовищі просто не набирається достатньо записів, щоб кнопка "Показати ще" з'явилася сама по собі.
- Прямі запити до API коректно відхиляють невалідні і чужі дані потрібними кодами: 401 без сесії, 403 при спробі скасувати чуже бронювання (і для одиничного бронювання, і для цілої серії), 404 на неіснуючу кімнату, 400 зі зрозумілим повідомленням за кожним правилом. Жодної 500 на некоректних або ворожих запитах не зустрілося.
- Часовий пояс: під браузером з іноземним поясом сітка і список "Мої бронювання" перебудовуються під пояс глядача, при цьому підпис прямо пояснює, що офіс працює за київським часом; межа робочого вікна перевірена і з цим зсувом - усе сходиться до хвилини.
- Бонусні сценарії: гонка при одночасному бронюванні (п'ятнадцять паралельних запитів на один слот - рівно одне успішне бронювання, решта чотирнадцять коректно відхилені, у базі один запис); повторювані серії бронювань (створення серії, скасування одного повторення без зачіпання решти, скасування всієї майбутньої серії однією дією); фільтр за місткістю кімнати; мобільна розкладка на ширині 390px, де сітка тижня перемикається на одноденний вигляд без горизонтальної прокрутки. Сповіщення про скорі закінчення бронювання також підтверджено окремою живою перевіркою: клієнт періодично опитує сервер, і коли наступний слот у кімнаті зайнятий, показується тост з назвою, кімнатою і часом (скріншот `33-notification-toast.png`).
- Обмеження на бронювання до підтвердження email відпрацювало надійно: щойно зареєстрований користувач отримує 403 зі зрозумілим повідомленням при спробі забронювати. Сам перехід за посиланням підтвердження (воно друкується в лог сервера, це описано в README) у межах цієї перевірки не відстежувався, тож увесь шлях від реєстрації до підтвердженого акаунта цілком не пройдено - але сама блокування для непідтверджених працює.

04 ЩО НЕ ПРАЦЮЄ АБО ЗЛАМАЛОСЯ

Некоректна дата в параметрі тижня може призвести до помилки сервера замість зрозумілої відмови. Параметр `weekStart` у ендпоінта списку бронювань кімнати перевіряється тільки за формою рядка (regex на чотири цифри року, дві на місяць, дві на день), без перевірки, що дата взагалі існує. Значення з неіснуючим місяцем (99-й місяць) проходить цю перевірку і перетворюється на невалідну дату, яка йде далі в запит до бази - очікувана відповідь тут 400 зі зрозумілим повідомленням, а по факту це виглядає як шлях до помилки 500. Місце в коді: `server/src/rooms/dto/week-bookings-query.dto.ts:4-7` (сама перевірка) і `server/src/rooms/rooms.service.ts:105-116` (куди невалідна дата потрапляє далі). Через звичайний інтерфейс це не зустріти - відтворюється прямим запитом до API з завідомо некоректною датою.

Скасувати бронювання із сітки можна тільки мишею. Блоки бронювань у тижневій сітці - це `div` з обробником кліку, але без ролі кнопки, `tabindex` і обробки клавіатури, тож користувач, що працює з клавіатури або з допоміжними технологіями, не може дійти до скасування через сітку - тільки через список "Мої бронювання", де кнопки зроблені як належить.

05 КОД І АРХІТЕКТУРА

Сильні сторони:

- Захист від подвійного бронювання продубльований на двох незалежних рівнях: предикат перетину в окремому модулі (`server/src/booking/overlap.ts:7`) ловить звичайний конфлікт, а обмеження `EXCLUDE USING gist` у самій базі (міграція `add_booking_exclusion_constraint`) ловить гонку при паралельних запитах і переводиться в ту саму зрозумілу 409. Це не теоретичний захист - жива перевірка з п'ятнадцятьма одночасними запитами підтвердила, що в базі лишається рівно один запис.
- Уся робота з часом прив'язана до часового поясу офісу в коді через `luxon`, а не до часового поясу процесу сервера: робочі години, кратність півгодини і підрахунок повторень серії рахуються в Europe/Kyiv незалежно від того, де фізично розгорнутий сервер, а зберігання - `timestampz`. Серії рахуються календарними тижнями в офісному поясі, тому переживають перехід на літній час.
- Акуратна робота з автентифікацією: паролі хешуються `argon2`, сесії живуть у базі з TTL і чисткою протухлих при логіні, кука в production виставлена як `httpOnly` + `sameSite=lax` + `secure`, а фіктивний хеш-пароль для неіснуючих email захищає від визначення зареєстрованих адрес за часом відповіді.
- Чиста шаруватість NestJS (контролер -> сервіс -> Prisma), DTO з `class-validator` і `whitelist: true`, єдиний формат помилок `{errors: {field|general}}` на весь бекенд. Сміття в коді майже немає: жодного `console.log`, `TODD`, `@ts-ignore` чи `any` на весь проект, коментарі тільки по справі.

Зони росту:

- `weekStart` варто перевіряти не тільки за формою рядка, а й на календарну валідність - подробиці і місце в коді див. у розділі "Що не працює" вище.
- Правила однієї й тієї самої сутності "бронювання" зараз розведені по двох сервісах: створення бронювання живе в `RoomsService` (`server/src/rooms/rooms.service.ts:140-247`), а скасування і списки - у `BookingService` (`server/src/booking/booking.service.ts:146-178`), при цьому спільний валідатор часу лежить у `booking/` і імпортується в `rooms/`. Варто або об'єднати ці правила в одному модулі, або явно розвести домени і дати кожному свій валідатор.
- Кука сесії виставляє `Expires`, порахований від серверного годинника, але без `Max-Age`. У звичайному браузері це не заважає входу - браузер звіряє термін життя куки з заголовком `Date` відповіді сервера і коректно враховує можливе розходження годинників, - але додавання `Max-Age` зробить час життя сесії стійким і не зав'язаним на цю компенсацію.
- У `docker-compose.yml` сервісу `server` не передається змінна `WEB_ORIGIN`, при цьому CORS налаштований з `credentials: true` (`configure-app.ts:12`). У поточній зв'язці все працює завдяки проксі Vite, але прямий запит до API на порт 3000 з іншим origin вже зламається - варто явно прокинути змінну в compose.

06 UI/UX

Що вдалося:

- Робота з часовими поясами - сильна сторона інтерфейсу: постійний підпис пояснює пояс глядача і робочі години офісу і на сітці, і в списку "Мої бронювання" (`17-tz-berlin-grid.png`, `27-tz-my-bookings.png`), та сама підказка повторюється прямо у формі створення бронювання поруч з полями часу, до того як

користувач помилились (`18-tz-boundary-rejected.png`).

- Діалог скасування бронювання - зразковий приклад патерну для незворотної дії: називає конкретне бронювання, прямо попереджає про незворотність, червона кнопка підтвердження проти нейтрального "Ні" за замовчуванням (`13-cancel-confirm.png` , `22-my-cancel-confirm.png`).
- Своє бронювання на сітці візуально відрізняється від чужого (підсвітка фону плюс ім'я автора як посилання), а не лишається неочевидним - саме така підказка про володіння потрібна для загального календаря (`04-grid-busy-slot.png`). Червона лінія поточного часу тримає постійну часову орієнтацію на будь-якому екрані розкладу (`03-grid-week.png`).
- Офлайн-стан і повідомлення про непідтверджений email у формі бронювання - зрозумілі, конкретні фрази українською, а не мовчазна відмова чи сирий код помилки (`29-unconfirmed-email-notice.png` , `32-submit-loading-offline.png`).
- Мобільна адаптація на ширині 390px зроблена не формально: сітка тижня перемикається на одноденний вигляд зі своїм перемикачем днів, форма і список вміщуються без горизонтальної прокрутки (`19-mobile-grid.png` , `20-mobile-form.png` , `21-mobile-my.png`).

Що покращити:

- Екран помилки завантаження даних прибирає всю навігацію сторінки - лишається гола стрічка з текстом і кнопка "Спробувати ще раз", без шапки і без шляху назад (`26-server-error-state.png`). Варто повернути навігацію на цей екран, щоб у користувача завжди був явний вихід.
- Обов'язкове поле "Назва" у формі створення бронювання не має власної валідації і покладається на нативне спливаюче повідомлення браузера - у середовищі перевірки воно показалося російською мовою (стандартна російська підказка браузера про обов'язкове поле) на тлі повністю українського інтерфейсу, що виглядає як явний збій локалізації, хоча формально це поведінка браузера, а не баг у коді (`30-empty-form-validation.png`).
- Порожній стан "Мої бронювання" для нового користувача обмежується текстом без явного заклику до дії - виявити шлях назад до списку кімнат можна тільки через невелике посилання в шапці (`28-empty-my-bookings.png`).
- Індикація відправлення форми під час очікування відповіді сервера - це тільки приглушення кнопки до 50% непрозорості, без спінера і без зміни тексту; на повільному з'єднанні цей стан легко не помітити (`31-submit-loading-throttled.png`).
- Клік по порожній комірці сітки не підставляє час у форму створення - модальне вікно завжди відкривається з порожніми полями часу, і користувачу доводиться вручну вводити той час, який він уже бачить у сітці.

07 РЕКОМЕНДАЦІЇ

- 1 Додати перевірку календарної валідності параметра `weekStart` на додаток до перевірки форми рядка, щоб некоректна дата віддавала зрозумілу 400, а не падала в запиті до бази.
- 2 Дати блокам бронювань у сітці роль кнопки, `tabindex` і обробку клавіатури, щоб скасування із сітки не вимагало обов'язково миші.
- 3 Додати `Max-Age` до сесійної куки поруч з `Expires`, а також явно прокинути `WEB_ORIGIN` серверу в `docker-compose.yml`, щоб CORS з `credentials: true` не тримався тільки на проксі Vite.
- 4 Звести правила однієї сутності "бронювання" (створення, скасування, списки) в одне місце замість розділення між `RoomsService` і `BookingService`.
- 5 Додати власну інлайн-валідацію поля "Назва" замість нативного браузерного повідомлення і повернути навігацію (шапку, шлях назад) на екран помилки завантаження даних.

- 6 Доповнити README таблицею змінних середовища, інструкцією з локального запуску без docker, коротким описом структури проекту і довідкою за основними ендпоінтами API - зараз ці розділи відсутні, при тому що опис механіки перетинів і зберігання часу в README вже зроблений докладно і по справі.

Робота зроблена на впевненому рівні, особливо в частині захисту від гонки при бронюванні і роботи з часовими поясами - обидві теми зазвичай і підводять подібні проекти, а тут витримані до дрібних граничних випадків. Перелічені пункти точкові і не вимагають переробки архітектури - усунення перших трьох уже помітно закрийє решту дір у надійності.